

Task vs function

SystemVerilog gives you several procedural shapes, and they are not interchangeable

Timing, return, and synthesis

- Here is the decision table in plain language
- Tasks may use hash-delay and at-events
- Functions must finish in zero simulation time and return a value
- Always_ff and always_comb are for synthesizable RTL
- The browser lab lets you flip between modes and see timing and synthesis roles side by

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Task vs function vs always

INTERACTIVE TOOL

Task vs function vs always

Separate **TB helpers** (task / function) from **RTL procedural blocks** (always) — timing, return values, and what synthesis will accept.

Starter example: a task that waits on clock — compare with a zero-time function and RTL always_* blocks.

Load starter example

Challenges (1/22)

Quiz: task. TB helper that may wait on clock is a? Answer: task

Answer

Show hint

Check

Next

Idle

quiz-task

quiz-function

quiz-always

quiz-synth

starter

preset-task

preset-function

preset-ff

preset-comb

run-task

run-fn

illegal

compare

demo

explain

mode-ff

code-task

code-fn

code-ff

prop-timing

reset-time

full-path

Core ideas

task

TB sequences — may # / @; no return value.

function

Zero-time computation; returns; OK in combo if pure.

always_*

Hardware processes — synthesis maps to FFs or gates.

Browser lab starter

learn_sv_tb — task vs function

Real SystemVerilog practice

- In the real SystemVerilog track, open this module's examples prompts
- Restate the core idea in one sentence , when you need to wait, use a task
- Sketch the four shapes on paper using the patterns below
- Optional stretch: mark which lines would fail synthesis if you moved them into RTL
- ```systemverilog // task , may wait`

Pitfalls to watch

- Do not put hash-delay or at-events inside a function
- Do not use a task where you only need a pure calculation
- Tasks describe testbench procedures
- And remember

Your turn

- Complete the checklist for at least one track , preferably both
- In the browser, load starter, compare task versus function timing
- On paper, write one task that waits two clock edges and one function that adds two bytes
- When you are ready, take the short quiz, then continue to fork and join in the next module